Simulating Quantum Hardware with Tidy3D and PhotonForge

A primer for the HardHaq starter notebooks

What this primer is for

The HardHaq challenges come with two Python notebooks: `Surface_Ion_Trap.ipynb` for the ion trap and `MZI_On_Chip.ipynb` for the Mach-Zehnder interferometer. Both run electromagnetic simulations with two Flexcompute tools:

- **Tidy3D**, an electromagnetic solver that runs on Flexcompute's cloud servers. It computes how electric and magnetic fields behave in and around a structure.
- **PhotonForge**, a design tool built on top of Tidy3D. You use it to draw devices as chip layouts, turn the layouts into 3D structures, simulate them, and connect simulated parts into circuits.

This primer explains the concepts you need before you open the notebooks. You do not need any experience with these tools, or with simulation in general. You do need basic Python (variables, functions, lists, loops) and some familiarity with Jupyter notebooks. The physics of each challenge is explained in the notebooks themselves; here the focus is on the tools.

Contents

1 Setting up **2**

1.1 Installing the software . . . 2

1.2 Connecting to the cloud . . . 2

1.3 Running a notebook . . . 2

2 How cloud simulation works **2**

2.1 Tasks . . . 2

2.2 FlexCredits and your budget . . . 3

2.3 Caching: running a cell twice is free . . . 3

3 Units and conventions **3**

4 What a simulation contains **4**

4.1 Materials . . . 4

4.2 The window . . . 4

4.3 The mesh . . . 4

4.4 Boundaries and symmetry . . . 5

5 The two solvers in the notebooks **6**

5.1 The mode solver 6

5.2 FDTD 7

6 PhotonForge: from layout to results **8**

6.1 Technology 8

6.2 Port specifications 9

6.3 Components and parametric components 10

6.4 Ports, references and routing 11

6.5 Models: how a component is simulated 12

6.6 The scattering matrix 12

7 A map of the starter notebooks **13**

7.1 Surface_Ion_Trap.ipynb 13

7.2 MZI_On_Chip.ipynb 14

8 Working effectively **14**

8.1 A good workflow 14

8.2 Writing a sweep 15

8.3 Common problems 15

9 Glossary **15**

1 Setting up

1.1 Installing the software

Both tools are Python packages. Installing PhotonForge also installs Tidy3D, because PhotonForge depends on it. In a terminal, preferably inside a fresh virtual environment, run:

```
pip install photonforge jupyterlab matplotlib
```

1.2 Connecting to the cloud

Simulations run on Flexcompute's servers, not on your laptop. Your computer only builds the simulation, sends it to the cloud and downloads the results. The servers need to know who you are, which is done with an **API key**: a long secret string linked to your account.

1. Sign in to your Flexcompute account (the organizers will tell you which account to use).
2. Copy your API key from the account settings page.
3. In a terminal, store the key on your computer:

   ```
   tidy3d configure --apikey=YOUR_API_KEY
   ```

Treat the API key like a password. Do not paste it into a notebook you share, and do not commit it to a public repository. Anyone with your key can spend your team's compute budget.

1.3 Running a notebook

Start JupyterLab from the folder that contains the notebooks:

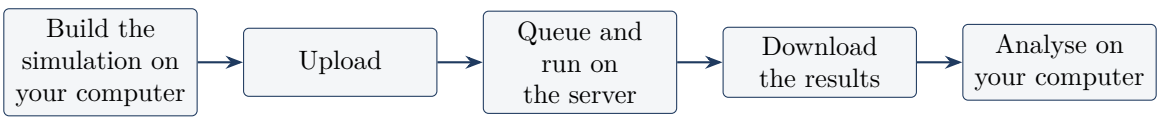
```
jupyter lab
```

Open a notebook and run the cells from top to bottom (Shift+Enter runs one cell). The cells depend on each other: each one uses variables made by the cells above it. If you change a parameter near the top, run all the cells below it again. If the notebook gets into a confusing state, use *Kernel, Restart Kernel and Run All Cells*.

2 How cloud simulation works

2.1 Tasks

Every simulation you send to the cloud is a **task**. A task goes through the same steps each time:



While a task runs, the notebook prints messages such as `Uploading task`, `Starting task`, a progress percentage, and `Downloading data`. It also prints a link to the task in the web interface, where you can see its status. The cell finishes when the results are on your computer.

2.2 FlexCredits and your budget

Simulations are paid for in **FlexCredits** (FC). Each team has a fixed budget. As a guide, from the starter notebooks:

Simulation	Typical time	Typical cost
Ion trap cross-section (mode solver)	about a minute	a small fraction of 1 FC
Waveguide or coupler cross-section (mode solver)	seconds	a small fraction of 1 FC
Directional coupler (FDTD)	1 to 2 minutes	about 0.5 FC
Full MZI (circuit, no new FDTD)	seconds	free

The MZI notebook asks the server for an **estimated cost** before it runs the FDTD simulation. The estimate is an upper bound: the starter coupler was estimated at 1.7 FC and billed 0.47 FC, because the simulation stops early once the fields have died away.

2.3 Caching: running a cell twice is free

PhotonForge remembers every result it has computed. If you run a simulation that is identical to one you already ran (same geometry, same materials, same frequencies, same settings), it reloads the stored result instead of starting a new task. This means:

- Re-running a notebook from the top after changing only a post-processing setting costs nothing.
- Going back to an earlier design costs nothing, as long as you use exactly the same values.
- Changing *anything* that enters the simulation, even by a tiny amount, creates a new task.

Spend the budget wisely

Before running a simulation, ask whether a cheaper calculation can answer the question first. Both notebooks contain a quick estimate (an analytic formula or a mode solve) that tells you roughly where a good design lies. Use it to choose which designs deserve a full simulation.

3 Units and conventions

Most mistakes in simulation scripts are unit mistakes. Tidy3D and PhotonForge use the same conventions:

Quantity	Unit
Length	micrometre (µm)
Frequency	hertz (Hz)
Time	second (s)
Angle	degrees in PhotonForge layout functions
Speed of light	`pf.C_0` or `td.C_0`, given in µm/s

Simulations are set up with **frequencies**, not wavelengths. Because lengths are in micrometres and the speed of light is in micrometres per second, converting is a single division:

```
wavelengths = np.linspace(1.530, 1.565, 36) # um
frequencies = pf.C_0 / wavelengths # Hz
```

The same holds for the ion trap: a 40 MHz drive is written `40e6`. Its wavelength in vacuum is `td.C_0 / 40e6`, about 7.5×10^6 µm, or 7.5 m.

Coordinates. A chip layout is drawn in the xy plane, as seen from above. The z axis points up, out of the chip. The top surface of the substrate is at $z = 0$.

4 What a simulation contains

Every simulation in the notebooks, whether a 2D cross-section or a 3D device, is built from the same few ingredients. PhotonForge assembles them for you from a layout and a technology (Section 6), but you need to know what they are to read the plots and to understand the settings the notebooks change.

4.1 Materials

A **medium** describes how a material responds to an electric field. The simplest one is set by its relative permittivity ε_r, the square of the refractive index n:

```
silicon = td.Medium(permittivity=3.48**2) # n = 3.48
silica = td.Medium(permittivity=1.44**2) # n = 1.44
air = td.Medium() # permittivity 1
```

A **perfect electric conductor** (`td.PEC`) is an ideal metal: the electric field inside it is zero, and its whole surface sits at one voltage. The ion trap notebook uses PEC for the gold electrodes. At the trap's radio frequencies this is accurate for the field shape, which is all the trap physics needs.

4.2 The window

A simulation only covers a finite box of space, the **window** or domain. Everything outside it is ignored. For a mode solve the window is a 2D rectangle in the cross-section; for FDTD it is a 3D box around the device. The window must be large enough that its edges do not change the answer. Light in a silicon waveguide is confined within about a micrometre, so a few micrometres are enough. The static field of the ion trap spreads much further: a 2 mm wide window made the trap depth 11 % too large, so the notebook uses an 8 mm window, where doubling the size changes the result by less than 1 %.

How to check a window

Enlarge the window and solve again. If the numbers you care about do not change, the window was large enough.

4.3 The mesh

The solver does not work with continuous space. It divides the window into small rectangular cells and computes the field at a few points in each cell. This set of cells is the **mesh** or **grid**.

- **Smaller cells give more accurate results** but cost more. In 3D, halving the cell size multiplies the number of cells by 8.
- **Cells must be small compared with the features that matter**: the wavelength inside the material, a narrow gap, or a region where the field changes quickly.

- **Automatic mesh.** PhotonForge chooses cell sizes from the wavelength and the materials, and refines them around metal edges and narrow gaps. In the MZI notebook, `grid_spec=12` means about 12 cells per wavelength inside the material.
- **Mesh override.** A `td.MeshOverrideStructure` forces small cells in a region you choose. The ion trap needs one: its RF wavelength is metres long, so the automatic mesh would put cells about 100 µm wide where the ion sits. The notebook sets 1 µm cells there (Figure 1).

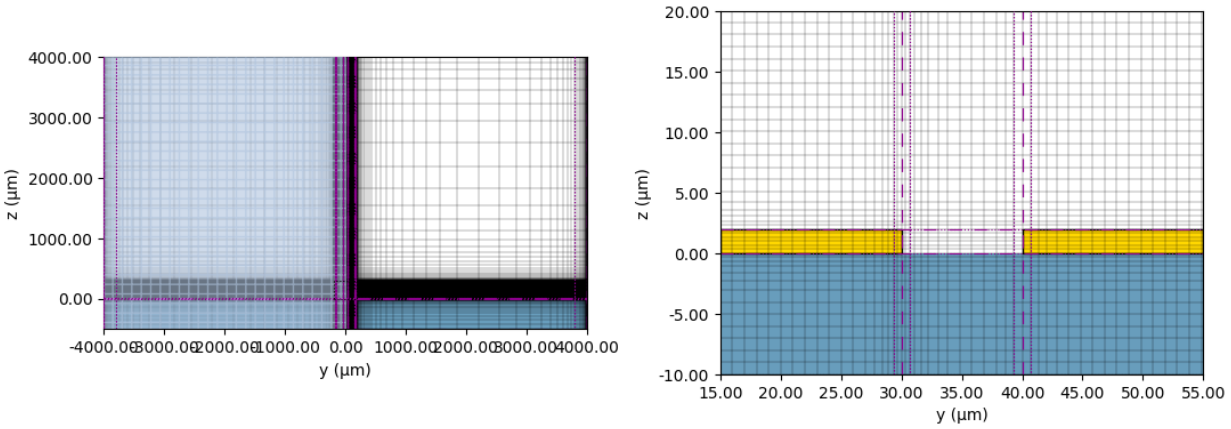


Figure 1: Mesh of the ion trap cross-section, from `Surface_Ion_Trap.ipynb`. Left: the whole 8 mm by 4.5 mm window. The cells are fine near the electrodes, in the dense band just above $z = 0$, and grow towards the edges. The shaded half ($y < 0$) is not solved, because of the mirror symmetry. Right: a close-up of one 10 µm gap between two electrodes (yellow) on the substrate (blue). The dashed lines mark the automatic refinement at the metal edges.

Keep the mesh fixed during a competition

Results change slightly when the mesh changes, so comparing two designs is only fair when both used the same mesh. The starter notebooks fix the mesh settings. Change the design parameters, not the mesh.

4.4 Boundaries and symmetry

The edges of the window need a rule. PhotonForge uses sensible defaults, and the notebooks keep them:

- **Mode solver: periodic edges.** The field leaving one edge comes back in at the opposite edge, as if the window repeated forever. Guided light never reaches the edges, so for a waveguide this choice does not matter. For the ion trap it is harmless as long as the window is large: periodic edges and grounded metal walls give the same trap depth to 0.1 % at the window size used.
- **FDTD: absorbing edges.** The edges absorb any light that reaches them, as if space continued forever, so light leaving the device does not bounce back. They appear as grey bands in the FDTD plots.

Symmetry. If the structure and the fields are mirror images of each other across a plane, only half of the window needs to be solved. This is set with `symmetry=(sx, sy, sz)`, where 0 means no symmetry along that axis and ±1 selects the type of mirror. The ion trap is symmetric about $y = 0$, and both RF electrodes carry the same voltage, so the notebook uses `symmetry=(0, 1, 0)` and solves only $y > 0$.

5 The two solvers in the notebooks

5.1 The mode solver

A **mode** of a waveguide is a field pattern that travels along it without changing shape. Its cross-section stays the same, and only its phase advances as it moves. The **mode solver** takes a 2D cross-section (a plane perpendicular to the direction of travel) and finds these patterns.

For each mode, the solver returns:

- the **field profile**: the electric and magnetic field across the plane;
- the **effective index** n_{eff}: the mode's phase advances by $2\pi n_{eff} L/\lambda$ over a length L, as if it travelled through a uniform material of index n_{eff};
- optionally the **group index** n_g, which sets how fast a pulse travels and how quickly the phase changes with wavelength.

In PhotonForge a mode solve is a single call to `pf.port_modes`, given a port specification (Section 6.2). The MZI notebook uses it on the strip waveguide:

```
strip_modes = pf.port_modes("Strip", frequencies, group_index=True)
n_eff = strip_modes.data.n_eff.isel(mode_index=0).values
```

A mode solve is cheap because it works in 2D. That makes it the right tool for quick estimates. The MZI notebook solves the cross-section of the two coupled waveguides to estimate the coupler length before any FDTD run (Figure 2).

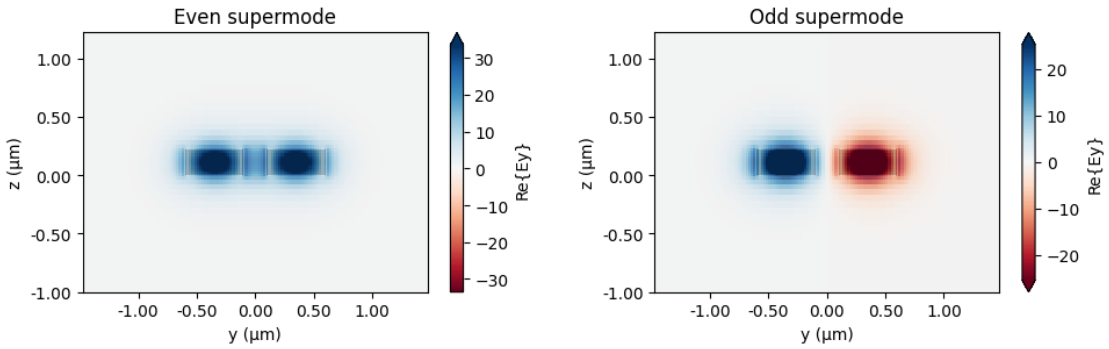


Figure 2: The two modes of a pair of coupled silicon waveguides, from `MZI_On_Chip.ipynb`. Red and blue are opposite signs of the field. In the even mode the field has the same sign in both waveguides; in the odd mode it has opposite signs. The small difference between their effective indices sets how fast light moves from one waveguide to the other.

The mode solver at radio frequencies. The ion trap is driven at 10 to 60 MHz, where the wavelength is metres long while the trap is a millimetre across. At this scale the field near the electrodes is the same as the static field of the electrodes held at fixed voltages, and it does not change along the length of the trap. The electrodes form a transmission line, and the mode solver finds its modes. Their transverse electric field is exactly the static field for a particular set of electrode voltages (Figure 3). The notebook then combines the modes so that the RF electrodes are at the drive voltage and the grounds are at 0 V. You do not need to change this step, but it helps to know why it is there.

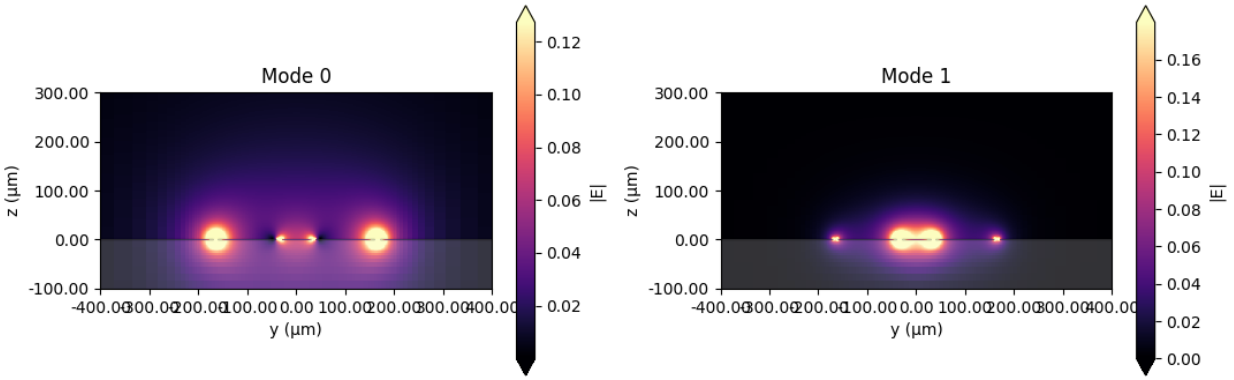


Figure 3: Electric field magnitude of the two modes of the trap electrodes, from `Surface_Ion_Trap.ipynb`. Each mode corresponds to a different set of electrode voltages. Neither is the trap field on its own; the notebook adds them with the right weights.

5.2 FDTD

The **finite-difference time-domain** (FDTD) method simulates light as it moves through a 3D structure. It launches a short pulse of light into an input, then steps Maxwell's equations forward in time on the mesh until the light has left the device. Recording what arrives at each output gives the response at every frequency in the pulse's band from a single run (Figure 4).

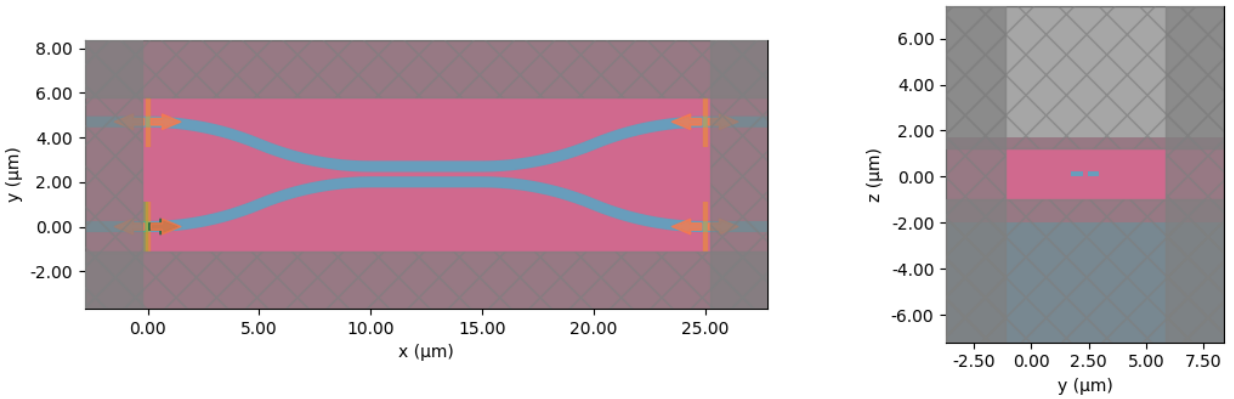


Figure 4: FDTD simulation of the directional coupler, from `MZI_On_Chip.ipynb`. Left: top view. The orange lines with arrows are the four ports, where light is launched (at the lower left port) and where the light leaving the device is recorded. The grey bands at the edges are absorbing boundaries. Right: side view showing the silicon layer inside the silica, with air above and the silicon substrate below.

FDTD is accurate and general, but it is the most expensive simulation in the challenges. Its cost grows with:

- the volume of the window (a longer coupler costs more),
- the fineness of the mesh,
- the simulated time, which is longer for devices that hold light for a long time.

Its cost does *not* grow much with the number of frequencies you ask for, since one pulse covers the whole band.

6 PhotonForge: from layout to results

PhotonForge organizes a design the way a chip designer thinks about it. A *technology* says how each drawing layer becomes a 3D structure; *port specifications* describe the waveguide cross-sections; a *component* is a 2D layout with ports; a *model* says how to compute what the component does. Figure 5 summarizes the flow.

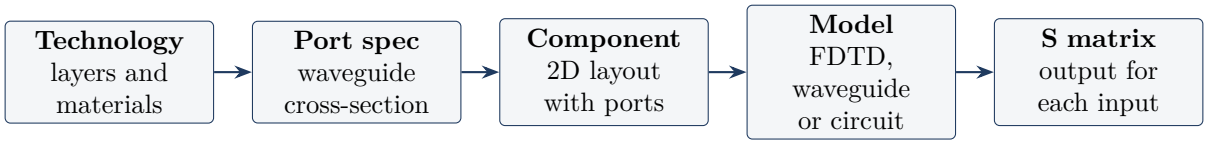


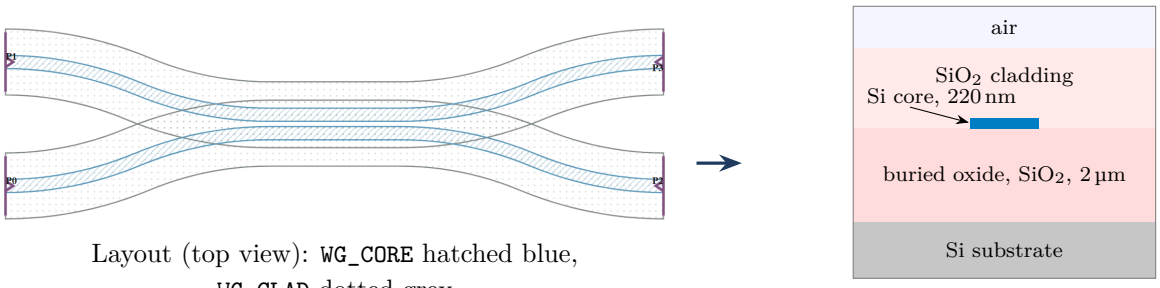
Figure 5: How PhotonForge turns a design into results.

6.1 Technology

A **technology** describes a fabrication process. It has three parts:

- **Layers**: named drawing layers, such as `WG_CORE` for the silicon waveguide core or `METAL` for the trap electrodes.
- **Extrusion specifications**: rules that turn the shapes on each layer into 3D, by giving a material and a range of heights. A rule with an empty mask (`pf.MaskSpec()`) applies everywhere, which is how blanket layers such as the substrate are made.
- **Port specifications**: the standard waveguide cross-sections of the process (Section 6.2).

Figure 6 shows what this means for the silicon technology of the MZI notebook. You draw only the top view. The technology fills in the vertical structure: wherever `WG_CORE` is drawn, a 220 nm silicon core appears on the buried oxide, and everything is covered in silica.



Layout (top view): `WG_CORE` hatched blue, `WG_CLAD` dotted grey

Extruded cross-section

Figure 6: A technology turns a 2D layout into 3D. Left: the directional coupler as drawn, with the silicon core on layer `WG_CORE`. Right: the cross-section that the extrusion rules of `pf.basic_technology` produce under every core shape (not to scale).

The MZI notebook uses this built-in technology, created with

```
technology = pf.basic_technology(strip_width=width)
```

so the waveguide width you choose sets the width of its `"Strip"` port. The ion trap notebook builds a small technology of its own: one `METAL` layer extruded as PEC from $z = 0$ to 2 µm, on a fused-silica substrate.

Most PhotonForge functions use the **default technology** unless told otherwise, which is why both notebooks set it once near the top:

```
pf.config.default_technology = technology
```

6.2 Port specifications

A **port specification** (`pf.PortSpec`) describes the cross-section of a waveguide or transmission line: which layers it is made of, where they sit, and the window used to solve for its modes. It is what the mode solver works on, and every port of a component refers to one. Its main arguments are shown in Figure 7:

- `width`: the size of the solver window across the waveguide, along y.
- `limits`: the bottom and top of the window, along z.
- `path_profiles`: one `(width, offset, layer)` entry for each strip of material in the cross-section. `width` is the strip's width, `offset` is the position of its centre measured from the middle of the window, and `layer` is the drawing layer it belongs to. The technology supplies the thickness and the material of that layer.
- `num_modes`: how many modes to find.
- `target_neff`: a starting guess for the effective index, usually the index of the core material.

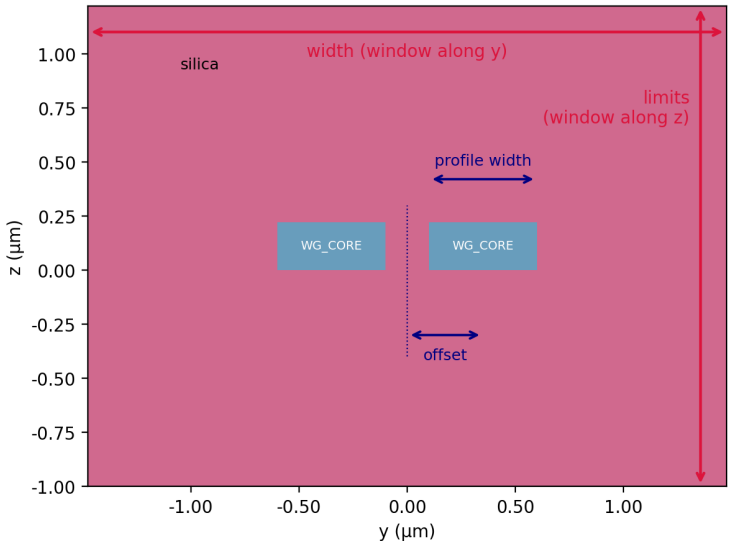


Figure 7: Anatomy of a port specification: the two coupled waveguides that the MZI notebook solves for its coupling length estimate, drawn with `pf.tidy3d_plot`. Red: the window, set by `width` and `limits`. Blue: one of the two `WG_CORE` path profiles, 0.5 µm wide, with its centre 0.35 µm from the middle of the window. A third, wider `WG_CLAD` profile fills the window with silica.

The code for this cross-section is:

```
coupled_strips = pf.PortSpec(
    description="Coupled strips",
    width=strip.width + coupling_distance, # window along y
    limits=strip.limits, # window along z
    num_modes=2,
    target_neff=3.5,
    path_profiles=[
        (width, coupling_distance / 2, "WG_CORE"), # upper core
        (width, -coupling_distance / 2, "WG_CORE"), # lower core
```

```
        (strip.path_profiles[1][0] + coupling_distance, 0, "WG_CLAD"),
    ],
)
```

Always plot a new port specification before solving it, with `pf.tidy3d_plot(spec, frequency=f)`. It shows exactly what the solver will see.

Electrical port specifications. The same object describes metal transmission lines. Adding a `voltage_path` (a line between two conductors) marks the port as electrical. The ion trap notebook describes its five electrodes this way, one path profile per electrode (Figure 8):

```
trap_spec = pf.PortSpec(
    description="Five-wire trap",
    width=2 * half_width,
    limits=(-substrate_thickness, top_height),
    num_modes=2,
    target_neff=1.5,
    path_profiles={
        "center": (center_width, 0, "METAL"),
        "rf0": (rf_width, y_rf, "METAL"),
        "rf1": (rf_width, -y_rf, "METAL"),
        "ground0": (outer_width, y_outer + outer_width / 2, "METAL"),
        "ground1": (outer_width, -y_outer - outer_width / 2, "METAL"),
    },
    voltage_path=[(y_outer, metal_thickness / 2), (y_rf, metal_thickness / 2)],
)
```

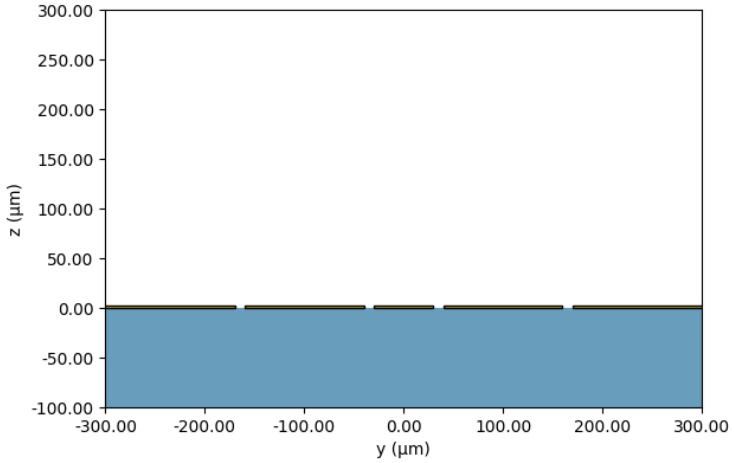


Figure 8: The trap port specification plotted with `pf.tidy3d_plot`, zoomed to the centre of its 8 mm window. From left to right: outer ground, RF rail, central ground, RF rail, outer ground, all 2 µm thick on the fused-silica substrate (blue).

A port specification turns into a mode solver with `spec.to_tidy3d(frequencies)`. The trap notebook does this so that it can add the mesh override and the symmetry before calling `pf.port_modes`.

6.3 Components and parametric components

A **component** is a device: a set of shapes on layers, plus ports and models. Figure 6 (left) is a component, the directional coupler; the purple triangles marked `P0` to `P3` are its ports. PhotonForge ships ready-made components in `pf.parametric`, such as straight waveguides, bends, S-bends and the S-bend coupler used in the MZI notebook.

A **parametric component** is a Python function, marked with the `@pf.parametric_component` decorator, that builds a component from a few numbers. Calling it with new values builds a new version of the device. This is how the MZI notebook exposes its design parameters:

```
@pf.parametric_component
def directional_coupler(*, gap=0.2, coupling_length=5.0, s_bend_length=10.0, s_bend_offset
    =2.0):
    ... # build the coupler from the arguments
    return pf.parametric.s_bend_coupler(...)

coupler = directional_coupler(gap=0.15, coupling_length=12.0)
```

Seeing a layout. The MZI notebook starts a **LiveViewer**, a layout viewer that runs in your browser. `viewer(component)` shows a component there and redraws it when you build a new version.

6.4 Ports, references and routing

A **port** is a place where light enters or leaves a component, together with its port specification. Ports are named `P0`, `P1`, `P2`, and so on. For the directional coupler: `P0` bottom left (input), `P1` top left, `P2` bottom right (through), `P3` top right (cross).

Larger devices are built from smaller ones. A **reference** places a copy of one component inside another. Two references of the same component share its simulation results, so the MZI uses the coupler twice but simulates it only once.

Waveguides between references are drawn by a **router**. `pf.parametric.route_auto` finds paths between pairs of ports on its own, and a `pf.PathLengthMatching` group sets a length target: the router equalizes `offset + routed length` over the routes, adding a meander of bends where one route has to be longer.

```
splitter = mzi.add_reference(coupler) # first coupler at the origin
combiner = mzi.add_reference(coupler) # second coupler ...
combiner.translate((splitter["P2"].center[0] + 150, 0)) # ... 150 um further on

arms = pf.parametric.route_auto(
    port1=[(splitter, "P2"), (splitter, "P3")],
    port2=[(combiner, "P0"), (combiner, "P1")],
    radius=10,
    matching=[pf.PathLengthMatching(offsets=[delta_length, 0.0])],
)
mzi.add_reference(arms)
mzi.add_port([splitter["P0"], splitter["P1"], combiner["P2"], combiner["P3"]])
```

Crediting the lower arm with an offset of ΔL makes the router lengthen the upper arm by exactly ΔL (Figure 9). The router does not report success, so the notebook measures both arms with `pf.route_length` and prints the difference it achieved.

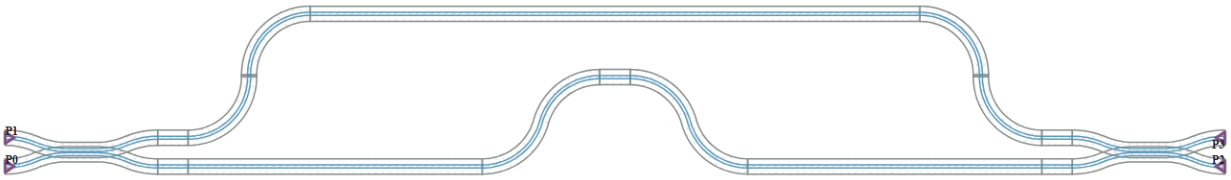


Figure 9: Layout of the starter MZI, from `MZI_On_Chip.ipynb`. Two directional couplers (left and right) are joined by two routed arms. The router shaped both paths so that the upper arm is exactly $\Delta L = 10\,\mu\text{m}$ longer than the lower one.

6.5 Models: how a component is simulated

A **model** attached to a component tells PhotonForge how to compute the component's response. The notebooks use three:

Model	What it does
`pf.Tidy3DModel`	Builds a 3D FDTD simulation from the layout and the technology and runs it on the cloud. Accurate, and the most expensive. Used for the directional coupler.
`pf.WaveguideModel`	Treats a waveguide piece (straight or bend) as a phase delay: it mode-solves the port cross-section and multiplies by the length. Cheap. The router gives every routed waveguide this kind of model.
`pf.CircuitModel`	Combines the results of all the references inside a component, following how their ports connect. Needs no new FDTD. Used for the whole MZI.

Port symmetries. An FDTD run launches light into one port at a time, so a four-port device would normally need four runs. If the device is mirror symmetric, some runs are copies of others with the ports renamed. `pf.Tidy3DModel(port_symmetries=...)` lists these renamings, and PhotonForge then runs only what it needs. For the symmetric directional coupler, one run with light entering `P0` gives everything.

6.6 The scattering matrix

Every model produces the same kind of result: a **scattering matrix**, or S matrix. For each pair of ports and each frequency it holds a complex number, the amplitude and phase of the light that arrives at one port when light of amplitude 1 enters another. The **power** transmitted is the squared magnitude.

```
s = coupler.s_matrix(frequencies) # runs the model (or loads the cache)
through = np.abs(s["P0@0", "P2@0"]) ** 2 # power from P0 to P2, at every frequency
cross = np.abs(s["P0@0", "P3@0"]) ** 2 # power from P0 to P3
```

The keys have the form `"port@mode"`: `"P0@0"` means mode 0 (the fundamental mode) at port `P0`. The first key is the input and the second is the output. In a lossless device the output powers for one input add up to 1; any shortfall is loss, for example light radiated away in a bend.

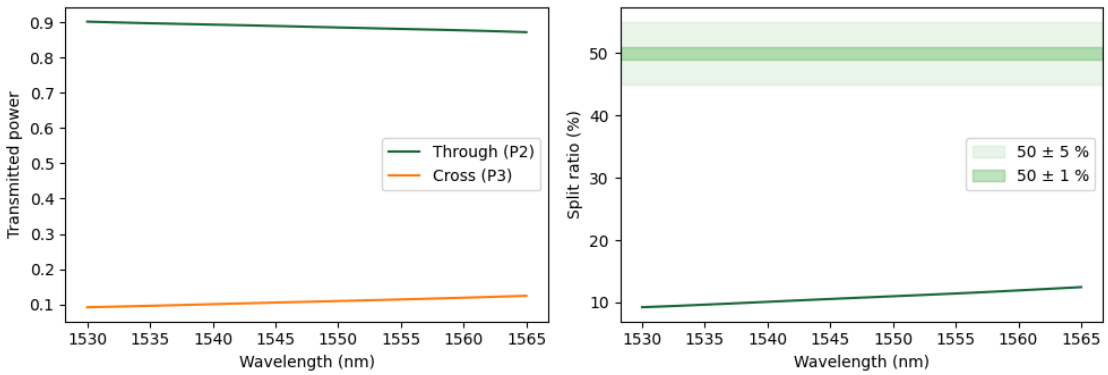


Figure 10: Through and cross power of the starter directional coupler across the band, computed from its S matrix (left), and the split ratio, the share of the output power in the cross port (right). The starter coupler sends only about 11 % to the cross port at 1550 nm; the shaded bands are the challenge targets.

7 A map of the starter notebooks

Each notebook is a linear script: parameters at the top, then building, simulating, post-processing and scoring. The tables below link each step to the concepts above.

7.1 Surface_Ion_Trap.ipynb

Step	What happens and which concepts it uses
Design parameters	The four numbers you tune: `rf_width`, `gap`, `v_rf`, `f_rf`.
Ion height estimate	A closed-form formula for the ion height. No simulation, free.
Technology	One `METAL` layer extruded as PEC on a fused-silica substrate.
Trap cross-section	An electrical port specification with one path profile per electrode, in an 8 mm window.
RF field	The port specification turned into a mode solver, with a mesh override over the ion region and mirror symmetry, solved with `pf.port_modes`. One cloud task.
Field with the RF rails at 1 V	Combines the two modes into the trap field. Post-processing only.
Pseudopotential and metrics	Converts the field into the pseudopotential and finds ion height, secular frequency, Mathieu q and trap depth. Post-processing only.
Score	Checks the constraints and returns the trap depth.

Changing `v_rf` only changes the post-processing, so it never needs a new task. Changing the geometry or the frequency does.

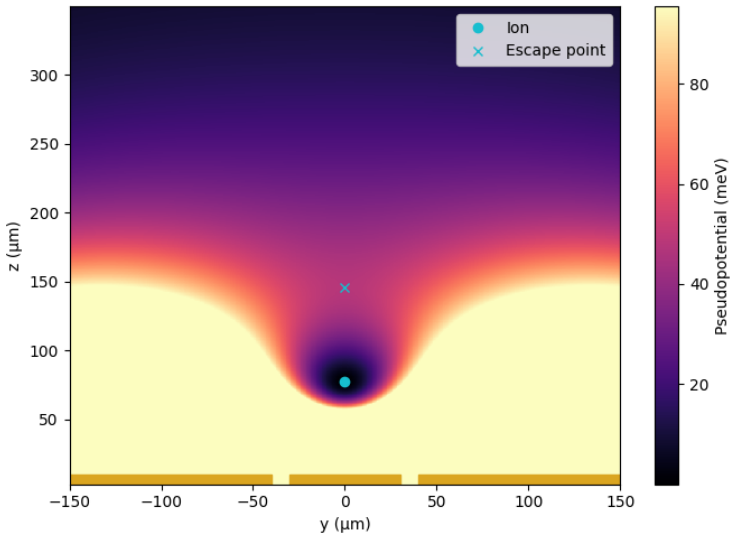


Figure 11: Pseudopotential above the electrodes for the starter trap. The ion sits at the dark minimum, about 75 µm above the surface. The cross marks the escape point, whose height above the minimum is the trap depth, 47.7 meV for the starter design.

7.2 MZI_On_Chip.ipynb

Step	What happens and which concepts it uses
Design parameters	The four numbers you tune: `width`, `gap`, `coupling_length`, `delta_length`.
Technology and waveguide mode	The built-in silicon technology with your waveguide width; one mode solve for n_{eff} and n_g.
Coupling length estimate	A port specification of the two coupled waveguides and one mode solve; gives the length for a 50:50 split without the S-bends. Cheap.
Directional coupler	A parametric component wrapping `pf.parametric.s_bend_coupler`, with a `Tidy3DModel` and port symmetries. The notebook plots the FDTD simulation and prints its estimated cost, then runs it. One FDTD task.
Mach-Zehnder interferometer	Two references of the coupler joined by `route_auto` with a `PathLengthMatching` target, combined by a `CircuitModel`. Reuses the coupler result.
Score	Checks the constraints and returns the extinction ratio, plus the stretch bonus.

Changing `delta_length` only changes the routing and the circuit, so it never needs a new FDTD run. Changing the width, gap or coupling length does.

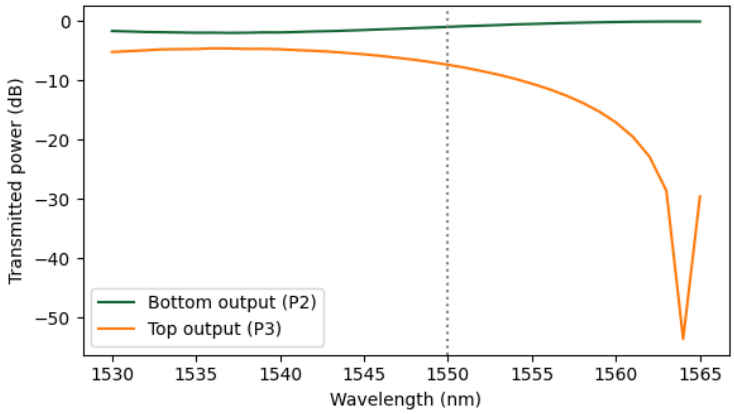


Figure 12: Power at the two MZI outputs for the starter design. The interference fringes are clear, but because the starter coupler is far from 50:50, the dark output at 1550 nm is far from dark.

8 Working effectively

8.1 A good workflow

1. **Run the starter notebook unchanged first.** Make sure everything works, and read every plot.
2. **Understand the trends with free or cheap tools.** Use the formulas in the notebook primers, the ion height estimate and the coupler length estimate.
3. **Plan a small set of simulations**, for example three coupling lengths, and interpolate between them instead of guessing one value at a time.

4. **Look at the geometry plot before every expensive run.** A wrong parameter is much cheaper to spot in a plot than in a result.
5. **Keep a record** of each design you simulated and its result, in a list or a small table in the notebook. Thanks to caching, re-running a recorded design costs nothing.

8.2 Writing a sweep

To compare many designs, wrap the steps from the layout to the score in a loop. For example, for the coupler:

```
results = []
for length in [10.0, 12.0, 14.0]:
    coupler = directional_coupler(gap=gap, coupling_length=length)
    s = coupler.s_matrix(frequencies)
    through = np.abs(s["P0@0", "P2@0"]) ** 2
    cross = np.abs(s["P0@0", "P3@0"]) ** 2
    results.append((length, 100 * cross[i_1550] / (through[i_1550] + cross[i_1550])))
```

Every new value in such a loop is a new task, so check the cost of one run first and multiply.

8.3 Common problems

Symptom	Likely cause
Authentication or API key error	The key is not configured on this computer; run `tidy3d configure` again.
`NameError` for a variable	A cell above was not run. Run all cells from the top.
Results did not change after editing a parameter	The cells below the parameter were not re-run.
A result looks wrong	Check units (micrometres, hertz) and look at the geometry plot.
The achieved ΔL differs from the target	The router could not fit the meander and left the arms unmatched. Keep ΔL inside its allowed range.
A task stays queued	The cloud is busy; the cell will continue once the task starts. Do not start the same task again in another cell.
The score is zero	At least one constraint fails; the score cell prints which one.

9 Glossary

API key
A secret string that identifies your account to the cloud servers.

Boundary condition
The rule applied at the edges of the window, such as periodic edges (mode solver) or absorbing edges (FDTD).

Caching
Reusing a stored result when an identical simulation is requested again.

Component
A PhotonForge device: shapes on layers, ports and models.

Effective index (n_{eff})
The index of a uniform material in which a plane wave would accumulate phase at the same rate as the mode.

Extrusion
Turning a 2D layout shape into a 3D structure with a given material and thickness.

FDTD
Finite-difference time-domain: a solver that steps Maxwell's equations forward in time on a 3D mesh.

FlexCredit (FC)
The unit in which cloud simulations are paid for.

Group index (n_g)
The ratio of the speed of light to the speed of a pulse in the waveguide.

Mesh (grid)
The division of the window into small cells.

Mode
A field pattern that travels along a waveguide without changing shape.

Mode solver
A solver that finds the modes of a 2D cross-section.

Model
A PhotonForge object that computes a component's S matrix.

Parametric component
A function that builds a component from a few parameters.

Path profile
One strip of material in a port specification: its width, the offset of its centre and its layer.

PEC
Perfect electric conductor: an ideal metal with no field inside it.

Port
A place where light enters or leaves a component, with a defined port specification.

Port specification
The description of a waveguide or transmission-line cross-section, with the window used to solve for its modes.

Reference
A placed copy of a component inside another component.

Router
A function that draws waveguides between ports, such as `route_auto`.

S matrix
The complex amplitude transmitted between every pair of ports at every frequency.

Symmetry
A mirror plane across which the problem repeats, used to solve only half of the window.

Task
One simulation sent to the cloud.

Technology
The description of a fabrication process: layers, materials, thicknesses and port specifications.

Window
The region of space a simulation covers.

Further reading

- PhotonForge documentation: `https://docs.flexcompute.com/projects/photonforge/`
- Tidy3D documentation: `https://docs.flexcompute.com/projects/tidy3d/`
- Flexcompute learning center: `https://www.flexcompute.com/tidy3d/learning-center/`